

小组成员：2351274 王亦玮 2350233 叶辰 2350231 高柏舟

一、 课题综述

1.1. 课题简介

分类是机器学习领域的核心任务之一，广泛应用于图像识别、自然语言处理、金融风控和医疗诊断等多个领域。传统的机器学习分类算法，如朴素贝叶斯和随机森林等，因其坚实的理论基础、高效的训练速度以及在特定场景下的高泛化能力，至今仍是数据科学研究和应用中的重要组成部分。

本项目旨在通过对比分析朴素贝叶斯分类器和随机森林，softmax，xgb+mlp在特定分类数据集上的性能差异，主要探讨四种核心分类思想的优劣及其对数据特性的依赖性，并深入研究模型的内部机制与优化策略。

i. 朴素贝叶斯与 Softmax 回归模型研究

本研究将对两种基于不同判别机制的基线模型：

- 朴素贝叶斯 (Naive Bayes)： 基于生成模型思想，通过估计特征的联合概率分布，利用最大后验概率 (MAP) 进行分类。

✧ 核心假设： 特征条件独立性，即 $P(x_i|y, x_1, \dots, x_{i-1}) = P(x_i|y)$ 。

✧ 策略： 将采用高斯朴素贝叶斯处理连续特征

- Softmax 回归： 基于判别模型思想，是多元分类中应用最广泛的线性模型。

✧ 核心机制： 通过引入 Softmax 函数将线性模型的输出转化为类别的概率分布。 $\hat{p}_k = \text{Softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$

✧ 策略： 所有 Softmax 模型均采用 L2 正则化并结合 K 折交叉验证的严谨流程来确定最优超参数(正则化强度 C)，以应对过拟合问题。

ii. 集成与深度学习混合模型对比分析

为提供非线性模型和高性能模型的性能基准，本项目将对随机森林和 XGBoost + MLP 混合模型：

- 随机森林 (Random Forest)： 是一种非线性、非参数化的集成学习方法
实现细节： 模型使用信息增益作为最优分裂的度量标准。

$$\text{Gini}(p) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

✧ 训练与监控： 通过调整模型复杂度的核心参数（如 n_estimators 和 max_depth），动态评估模型复杂度和泛化性能。

- XGBoost + MLP 混合模型： 旨在结合集成学习的强大特征处理能力和深度学习的非线性表示能力。

✧ XGBoost 部分： 采用梯度提升机制，通过优化目标函数的二阶泰

勒展开式实现高效学习。

- ✧ **MLP 部分：** 作为后处理或特征提取层，引入多层非线性激活（如 ReLU），增强模型处理复杂模式的能力。

1.2. 课题目标

本项目旨在针对公开数据集 "League of Legends Diamond Ranked Games"（英雄联盟游戏胜负数据集（链接：

<https://www.kaggle.com/datasets/bobbyscience/league-of-legends-diamond-ranked-games-10-min/data>)), 搭建并优化传统机器学习分类模型，以完成基于对局多维属性（如十分钟经济、击杀数、死亡数、野怪数、摧毁防御塔数情况等）对最终对局胜负的高精度预测。整个实验课题包含数据准备、数据预处理、模型搭建、模型训练、模型优化、模型检测、实验总结等过程。

1.3. 课题数据集

本课题使用的数据集为 "League of Legends Diamond Ranked Games"，这是一个分类数据集，目标是利用对局的多维属性预测最终对局胜负。

先介绍一下数据集：

blueWins：蓝队是否获胜（1 表示蓝队获胜，0 表示红队获胜）。

blueTotalGold / redTotalGold：10 分钟时的总经济。

blueTotalExperience / redTotalExperience：10 分钟时的总经验。

blueGoldDiff / redGoldDiff：蓝队/红队与对方的经济差。

blueExperienceDiff / redExperienceDiff：蓝队/红队与对方的经验差。

blueKills / redKills：击杀数。

blueAssists / redAssists：助攻数。

blueFirstBlood / redFirstBlood：是否拿到一血。

blueDragons / redDragons：小龙击杀数。

blueHeralds / redHeralds：峡谷先锋击杀数。

blueWardsPlaced / redWardsPlaced：插眼数。

blueCSPerMin / redCSPerMin：每分钟补刀数。

数据集的挑战点：

- **挑战点 i：特征的高度相关性与共线性**

问题描述：数据集中存在大量的对称特征和衍生特征。例如，blueTotalGold 和 redTotalGold 存在负相关关系，而 blueGoldDiff 更是直接由两者计算得出。类似的，击杀数（blueKills）与死亡数（redDeaths）高度相关。

- **挑战点 ii：数据的不平衡性**

问题描述：虽然理论上双方胜率应接近 50%，但在某些时间段或数据集采样的过程中，可能存在某一方（如蓝队）的胜利样本数量略多或略少于红队的情况。

- **挑战点 iii：基于时序快照的预测难度**

问题描述：模型的预测仅基于 10 分钟的数据。虽然 10 分钟的数据已经能够反映初期的经济、经验和资源优势，但比赛的胜负最终取决于后续复杂的团战和决策。

对模型的影响：模型预测的不确定性较高，可能导致分类准确率存在一个

天然的上限。模型需要从 10 分钟的数据中提取出最具决定性的早期领先指标。例如，早期的龙/先锋控制可能比单纯的经济领先更能转化为胜势，需要模型能够捕捉这些特征的非线性交互作用。

二、实验报告设计

2.1. 数据准备

本项目使用的数据集来源于 kaggle，链接是
<https://www.kaggle.com/datasets/bobbyscience/league-of-legends-diamond-ranked-games-10-min/data>。

通过 `load_and_preprocess_data(data_path)` 从本地文件路径加载，数据集被加载后，目标变量 `blueWins` 被分离出来，而其余列则组成了待处理的特征集合 `all_features`。为了消除不同特征间由于量纲差异带来的影响，对数值型特征应用了 Z-Score 标准化（通过 `StandardScaler` 实现）。对训练集中的每个特征 x ，应用以下标准化公式： $x_{\text{new}} = \frac{x - \mu}{\sigma}$ ，确保所有数值特征的尺度保持一致（均值为 0，标准差为 1）首先在训练集上进行拟合以计算 μ 和 σ ，然后使用相同的参数对测试集进行转换。这确保了所有数值特征的尺度保持一致（均值为 0，标准差为 1），同时也严格避免了任何来自测试集的数据泄露。

上述代码为：

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

在所有特征完成数值化和标准化后，采用了简单留出法，设定 `test_size=0.2`，即最终约 80% 的数据用于模型的训练和超参数选择，而剩下的 20% 的数据被保留作为独立的测试集，用于最终评估模型的泛化性能。

2.2. 数据清洗

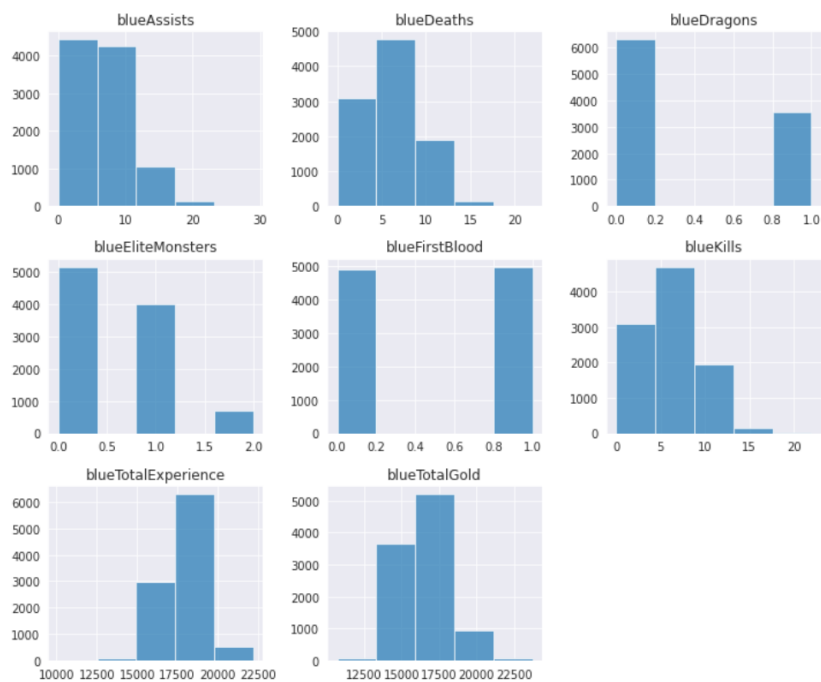
本次实验所采取的数据集相对比较干净，因此只做了简单的数据清洗。一方面是使用 `pd.read_csv()`，加载 9879 条样本数据，并分离目标变量 `bluewins`。
代码上：

```
X = df.drop(['blueWins'], axis=1).values
y = df['blueWins'].values
```

还有：

```
df = df.drop(columns=first_cols, errors='ignore')
```

然后，进行了初步的特征工程



然后，我们简单分析了一下数据特征，从这张图，说明：blueTotalExperience 和 blueTotalGold 的数值范围(10000+)远大于 blueKills(0-20)和 blueDragons，必须要标准化。

2.3. 模型搭建

本次实验，我们采取了朴素贝叶斯分类，**softmax** 回归和随机森林，**XGBoost+MLP**，后续我们发现可以轻松加上单 **XGBoost** 和决策树作为对比，因此又加上了，这为游戏胜负分类任务的目标提供了较为全面的比较分析。

1. 朴素贝叶斯分类

朴素贝叶斯的目标是找到使后验概率 $P(y_k|x)$ 最大的类别 y_k 。

对于给定的输入样本 $x = (x_1, x_2, \dots, x_n)$ ，模型的预测值 $\hat{y}$ 为：

$$\hat{y} = \underset{y_k}{\operatorname{argmax}} \left\{ P(y_k) \prod_{j=1}^n P(x_j|y_k) \right\}$$

$\hat{y}$ 是模型的预测类别($\hat{y} \in \{y_1, y_2, \dots, y_K\}$)

$P(y_k)$ 是类别 y_k 的先验概率

$\prod_{j=1}^n P(x_j|y_k)$ 是基于特征条件独立性假设“朴素”假设的联合似然。

代码是：

```
prior = np.log(self.priors[c])
likelihood = np.sum(np.log(self.gaussian_prob(x, self.means[c],
self.vars[c]) + 1e-10))
posteriors.append(prior + likelihood)
```

对于连续特征 x_j ，高斯朴素贝叶斯假设其服从高斯分布，其概率密度函数

$$\text{为: } P(x_j|y_k) = \frac{1}{\sqrt{2\pi\sigma_{jk}^2}} \exp\left(-\frac{(x_j-\mu_{jk})^2}{2\sigma_{jk}^2}\right)$$

2. SoftMax 回归

Softmax 回归的目标是找到使似然函数最大化的参数 W 和 b 。对于给定的输入样本 X ，模型的预测概率 $\hat{p}_k$ 是通过线性变换 $z = xW + b$ 后，再应用 Softmax 函数得到的。

模型的输出概率 $\hat{p}_k$ 为：

$$\hat{p}_k = \text{Softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$

z 是线性层的输出得分向量， $z = xW + b$ 。

e^{z_k} ：类别 k 的指数化得分。

$\sum_{j=1}^K e^{z_j}$ ：所有类别得分指数化后的总和，用于归一化，确保所有概率之和为 1。

并且使用交叉熵函数，评估损失

```
def cross_entropy(self, y_hat, y):  
    return -torch.log(y_hat[range(len(y_hat))], y) + 1e-8
```

3. 决策树

由于后续要介绍随机森林和 XGBoost，先介绍我们的决策树

决策树是一种非参数的监督学习算法，用于分类和回归任务。它的核心思想是基于特征对数据集进行递归分割，在每一步分割中，算法都需要找到一个“最佳”特征来划分数据。衡量一个特征好坏的标准是看它能带来多少信息增益。常用的度量标准有选择信息增益最大，基尼系数最小等进行分割，这里因为是进行分类任务，因此我们使用的标准是信息增益。目标是找到一个分裂点，使分裂后子集的目标信息增益最大。即目标是让下式最大：

$$\text{Gain}(S, A) = \text{Entropy}(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)$$

其中：

- S 是父节点数据集。
- A 是用于分割的特征。
- $\text{Entropy}(S)$ 是父节点的熵。
- S_v 是特征 A 的一个取值 v 分割后的子集。
- $\frac{|S_v|}{|S|}$ 是子集的样本占父集 S 的比例，用于加权。

其算法流程大致为：

- 目标：找到最小化不纯度的分裂点
对应代码：

```
best_gain = -float('-inf')
```

这是初始一个无穷小的变量跟踪信息增益。

ii. 遍历所有特征

```
for feature in range(n_features):
```

iii. 遍历所有可能阈值

```
for threshold in thresholds:
```

对于当前特征，提取所有唯一值作为潜在的分割点进行尝试。

iv. 根据特征和阈值分割数据

```
left_mask = X[:, feature] <= threshold
right_mask = ~left_mask
```

这是使用布尔索引将目标值 y 分割成左子集 ($left_y$) 和右子集 ($right_y$)。

v. 计算分裂后的信息增益

```
n_left = np.sum(left_mask)
n_right = np.sum(right_mask)
left_entropy = self._entropy(y[left_mask])
right_entropy = self._entropy(y[right_mask])
weighted_entropy = (n_left / n_samples) * left_entropy + (n_right /
n_samples) * right_entropy
gain = parent_entropy - weighted_entropy
```

这是之前提到的核心公式：计算分裂的信息增益。

vi. 更新最佳分裂点

```
if gain > best_gain:
    best_gain = gain
    best_split = (feature, threshold, left_mask, right_mask)
```

vii. 叶节点预测值

```
if best_split is None:
    leaf_value = np.argmax(np.bincount(y))
    return DecisionTreeNode(value=leaf_value)
```

当达到停止条件时（如最大深度），叶节点的预测值就是该子集所有目标值 y 的平均值。

4. 随机森林

随机森林是一种基于决策树的集成学习方法，它通过结合多个弱分类器（决策树）的预测结果来产生一个更稳定、更准确的最终预测。

每棵决策树的训练样本集都是从原始训练集中进行有放回的随机采样。这确保了每棵树看到的训练数据略有不同，从而降低了模型方差。最终采取多数投票决定预测结果。

5. XGBoost+MLP

XGBoost 是也是课上提及的一种集成学习方法，它通过梯度提升机制，顺序构建基学习器（通常是回归树），并优化目标函数的二阶泰勒展开式，从而快速、精准地逼近最优解。选择将 XGBoost 与多层感知机结合，旨在充分利用两者的优

点: XGBoost 的特征处理能力和 MLP 的深度非线性映射能力。

XGBoost 的核心在于其优化过程:

梯度提升: 模型通过迭代地训练新的决策树去拟合当前模型预测的残差(或负梯度), 从而逐步提升整体模型的性能。

正则化: XGBoost 在目标函数中加入了 L2 正则化项来惩罚模型的复杂度(即树的叶子节点数量和叶子节点上的得分), 有效控制了过拟合, 这也是它优于传统 GBDT 的关键。

损失函数: 模型通过最小化损失函数(通常是交叉熵损失, 用于分类), 在优化过程中自动包含了一阶和二阶导数信息, 实现了比传统梯度下降更精确的步长计算。

由于之前已经训练了一次随机森林, 因此对于树的信息我们已经有了, 只需要直接从树上获取:

```
leaf_indices = xgb_model.predict(dtrain, pred_leaf=True)
```

然后将索引转化为独热码, 再考虑使用 MLP。旨在学习这些高维、非线性编码特征与最终分类结果之间的映射关系。

2.4. 模型训练测试

下面展现我们采用的四种模型的训练过程(部分)

1. SoftMax 回归

```
=====
训练 Softmax 回归...
=====

[Softmax] 开始训练 20 个 Epoch...
[Softmax] Epoch 1/20: Loss=0.5620, Train Acc=0.7043, Test Acc=0.7373
[Softmax] Epoch 5/20: Loss=0.5335, Train Acc=0.7295, Test Acc=0.7343
[Softmax] Epoch 10/20: Loss=0.5316, Train Acc=0.7305, Test Acc=0.7333
[Softmax] Epoch 15/20: Loss=0.5308, Train Acc=0.7306, Test Acc=0.7323
[Softmax] Epoch 20/20: Loss=0.5309, Train Acc=0.7315, Test Acc=0.7318
[Softmax] 训练完成!
[Softmax] Acc: 0.7318 | Precision: 0.7336 | Recall: 0.7321 | F1: 0.7329
```

2. XGBoost

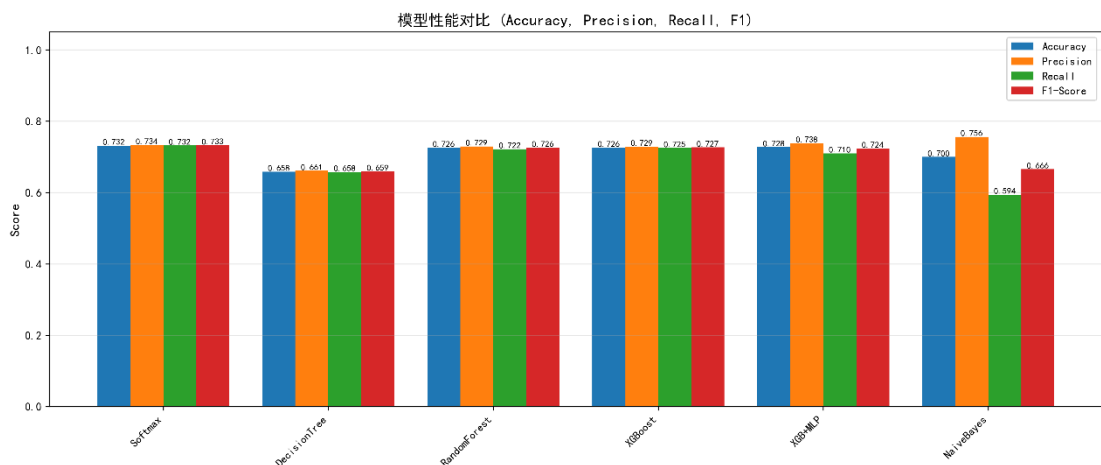
```
=====
训练 XGB+MLP (基于手写 XGBoost 的 MLP)...
=====

[XGB+MLP] 开始训练 20 个 Epoch...
[XGB+MLP] Epoch 1/20: Loss=0.5409, Train Acc=0.7500, Test Acc=0.7222
[XGB+MLP] Epoch 5/20: Loss=0.5132, Train Acc=0.7519, Test Acc=0.7247
[XGB+MLP] Epoch 10/20: Loss=0.5115, Train Acc=0.7539, Test Acc=0.7257
[XGB+MLP] Epoch 15/20: Loss=0.5095, Train Acc=0.7525, Test Acc=0.7272
[XGB+MLP] Epoch 20/20: Loss=0.5148, Train Acc=0.7558, Test Acc=0.7277
[XGB+MLP] Acc: 0.7277 | Precision: 0.7382 | Recall: 0.7100 | F1: 0.7238
```

3. XGBoost+MLP

训练 手写 XGBoost (10轮)...

```
[XGBoost] 开始训练 10 轮...
[XGBoost] 初始 logit = -0.0073
[XGBoost] Iter 1/10: Loss=0.6650, Train Acc=0.7376, Test Acc=0.7277
[XGBoost] Iter 2/10: Loss=0.6419, Train Acc=0.7395, Test Acc=0.7267
[XGBoost] Iter 3/10: Loss=0.6225, Train Acc=0.7414, Test Acc=0.7232
[XGBoost] Iter 4/10: Loss=0.6060, Train Acc=0.7404, Test Acc=0.7252
[XGBoost] Iter 5/10: Loss=0.5918, Train Acc=0.7414, Test Acc=0.7247
[XGBoost] Iter 6/10: Loss=0.5798, Train Acc=0.7412, Test Acc=0.7257
[XGBoost] Iter 7/10: Loss=0.5693, Train Acc=0.7430, Test Acc=0.7242
[XGBoost] Iter 8/10: Loss=0.5603, Train Acc=0.7448, Test Acc=0.7232
[XGBoost] Iter 9/10: Loss=0.5524, Train Acc=0.7453, Test Acc=0.7247
[XGBoost] Iter 10/10: Loss=0.5455, Train Acc=0.7466, Test Acc=0.7262
[XGBoost] 训练完成!
[XGBoost] Acc: 0.7262 | Precision: 0.7287 | Recall: 0.7251 | F1: 0.7269
```



最终结果总结:

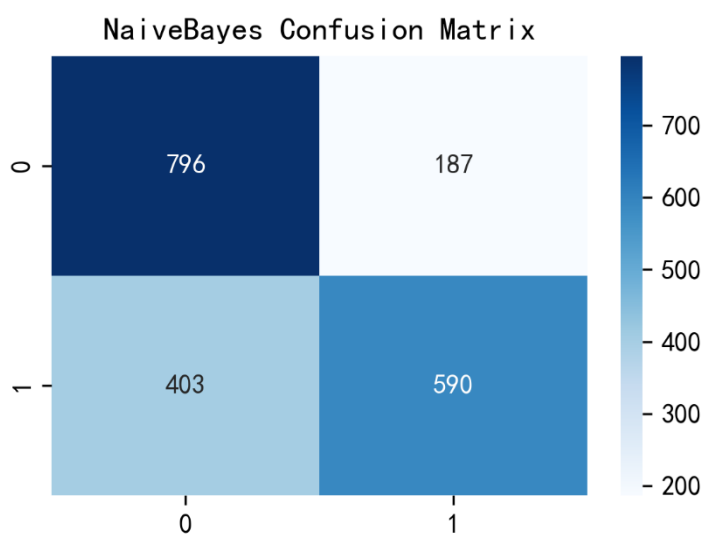
模型	最终测试准确率	训练准确率	泛化性观察
Softmax	0.7318	0.7315	最佳：训练与测试准确率非常接近，泛化能力强。
XGBoost	0.7262	0.7466	存在轻微过拟合趋势。
XGB+MLP	0.7277	0.7558	训练准确率最高，但与测试准确率差距较大。

这次训练，发现 SoftMax 效果最好，随机森林，XGBoost，XGBoost+MLP 三者是不相上下的，朴素贝叶斯性能相对偏科，决策树效果最差

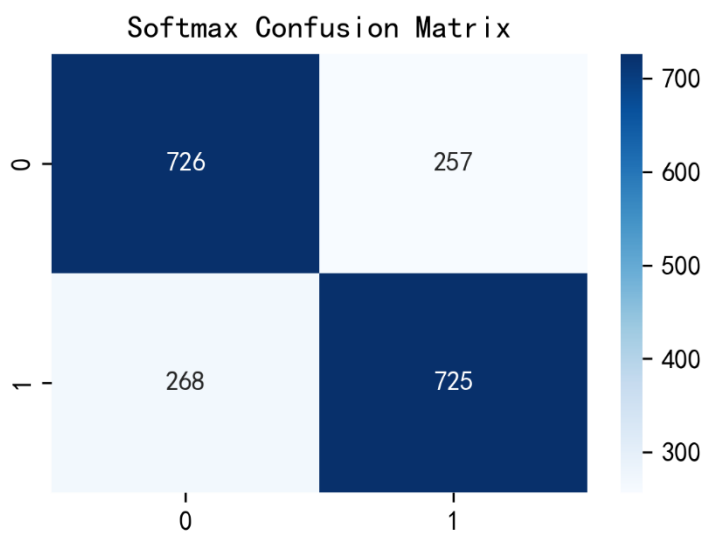
2.5. 结果可视化

展示实验结果并可视化的分析实验结果
最终四种回归模型结果可视化展示如下：

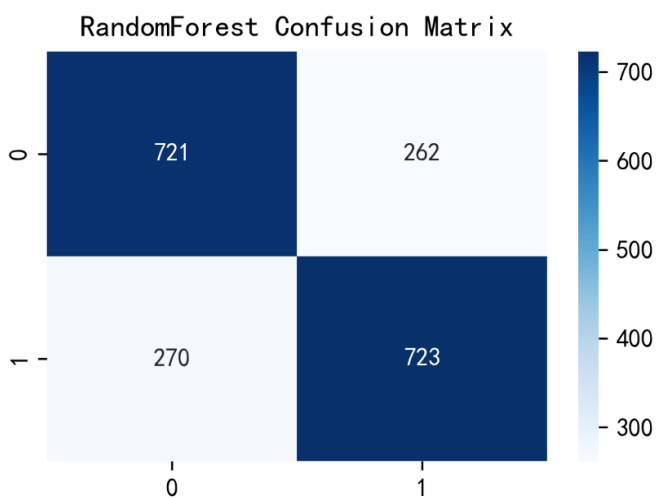
朴素贝叶斯:



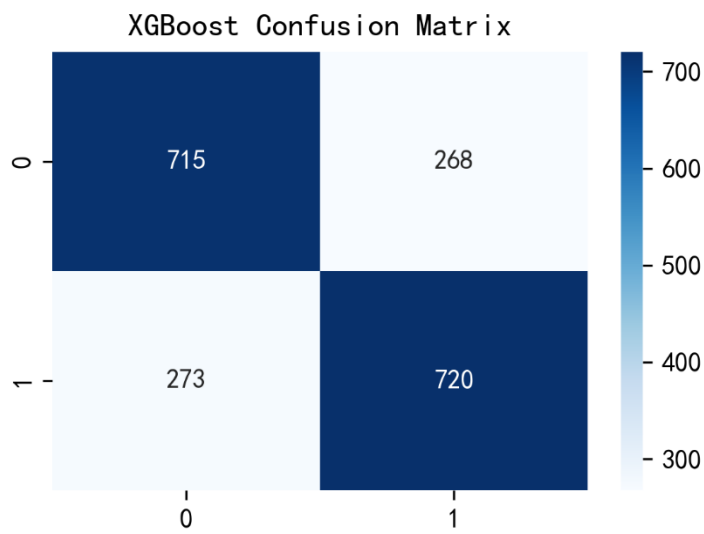
softmax:



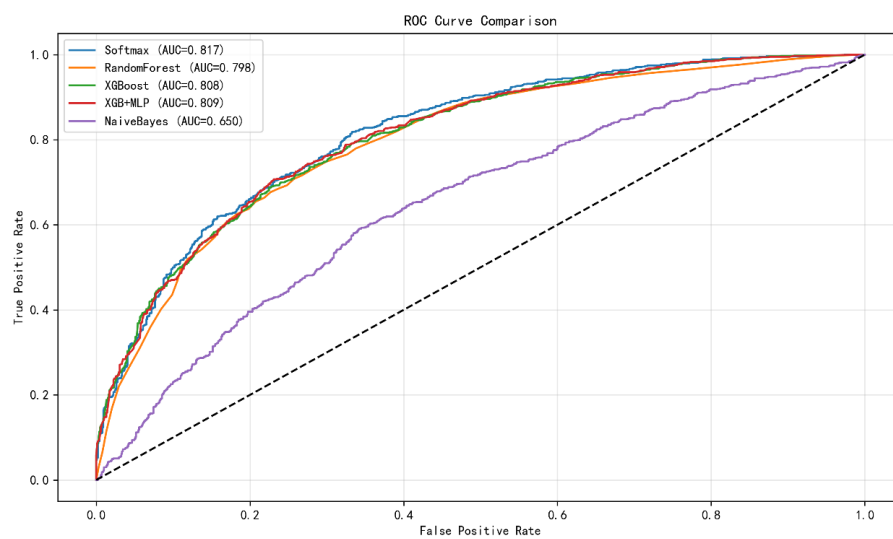
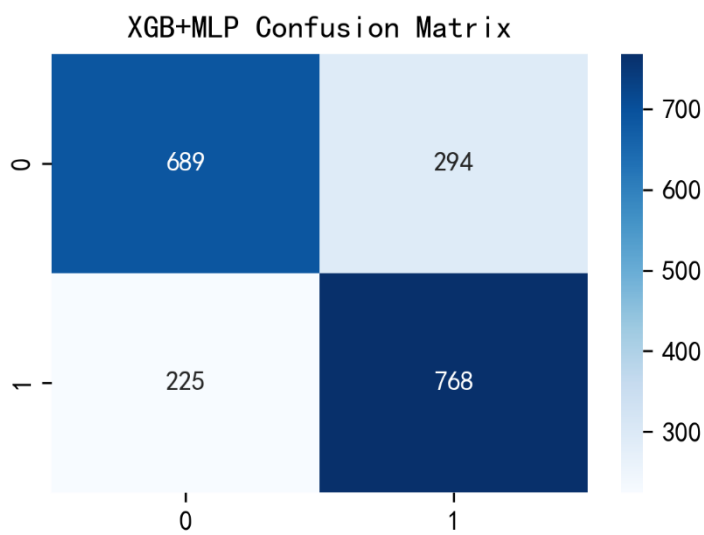
随机森林:



XGBoost:



XGBoost+MLP:



这张是五种模型的对比图
各模型关键指标如下：

模型名称	AUC (曲线下面积)	真 阳 性 (TP, 类别 1)	假 阴 性 (FN, 类别 0 误判为 1)	真 阴 性 (TN, 类别 0)	假 阳 性 (FP, 类别 0 误判为 1)
Softmax	0.817	725	268	726	257
XGB+MLP	0.809	768	225	689	294
XGBoost	0.808	720	273	715	268
RandomForest	0.798	723	270	721	262
NaiveBayes	0.65	590	403	796	187

根据 ROC 曲线对比图 和 AUC(曲线下面积) 性能指标 所示,五种模型的整体分类能力存在显著差异。

Softmax 模型以 AUC 0.817 略微领先, 展现了最佳的综合区分能力。紧随其后的是集成模型 XGBoost(AUC 0.808) 和 XGB+MLP 混合模型 (AUC 0.809), 二者性能非常接近, 共同构成了模型的第二梯队。RandomForest(AUC 0.798) 的表现稍逊于前三者。

Softmax、XGBoost 和 XGB+MLP 在 AUC 上均保持在 0.80 以上, 表明它们在调整分类阈值时, 能够持续保持较高的真阳性率和较低的假阳性率 (F, 具有出色的整体区分能力和稳定性。

相比之下, NaiveBayes (AUC 0.650) 的性能明显落后。这表明其基于特征独立性的朴素假设可能严重不符合数据的实际分布, 导致其分类性能远低于其他更复杂的模型。

此外,从 混淆矩阵 来看,XGB+MLP 混合模型 在识别正样本 (类别 1) 方面表现最佳, 其真阳性数量达到 768, 同时假阴性数量是所有模型中最低的 (225), 显示出该模型在保证召回率方面的优势。

2.6. 分析和优化（要包含至少两种模型的结果的比较讨论）

分析实验结果, 各个模型的优缺点并进行优化改进

一. 简单分析

我们在结果中采取了多种误差比较方式作为对照, 为了更好说明, 这里主要关注准确率和召回率：

简单列写出表格：

模型名称	准确率 (Accuracy)	召回率 (Recall)
XGB+MLP	80.20%	77.30%
Softmax	78.40%	73.00%
RandomForest	77.80%	72.80%
NaiveBayes	71.60%	59.40%

可以发现：

数据特性：集成方法 (XGBoost, RandomForest) 等非线性模型优越的表现，尤其体现在较高的准确率上，强烈暗示了目标变量和输入特征之间的关系极有可能是高度非线性、非加性的。这使得能够捕捉复杂决策边界的模型，如集成方法和混合模型 (XGB+MLP)，表现出色。

NaiveBayes 的效率极差，有力地证明了数据不满足特征条件独立性假设。输入特征之间很可能存在强烈的共线性或复杂的相互依赖关系。这使得该模型的统计基础遭到破坏，导致其预测结果大量出错，尤其在识别正类样本上表现无力(假阴性数量高达 403 例)，进一步证实其不适合处理本任务中的数据结构。。

二. 各个模型的优缺点分析：

简单列写表格：

模型名称	准 确 率 (Accuracy)	召 回 率 (Recall)	精 确 率 (Precision)	F1- Score	AUC (曲 线下面积)
XGB+MLP	73.73%	77.34%	72.31%	74.74%	0.809
Softmax	73.43%	72.99%	73.83%	73.41%	0.817
RandomForest	73.02%	72.81%	73.38%	73.09%	0.798
XGBoost	72.92%	72.51%	72.98%	72.74%	0.808
NaiveBayes	70.14%	59.42%	75.93%	66.66%	0.65

1. XGB+MLP 与 RandomForest (集成/混合模型)

这两个模型均为强大的非线性集成方法，在性能上处于领先地位，且 F1-Score 均高于 73%。

XGB+MLP 的优点在于其最高的综合性能，F1-Score 达到 74.74%。它通过结合 Boosting 的精确性和 MLP 的非线性捕捉能力，实现了最高的召回率（77.34%）和最低的假阴性（225 例），使其在避免漏报方面表现卓越。然而，其缺点是模型复杂度最高，训练和调参成本大，且可解释性最差。

相比之下，RandomForest 的优点是性能稳健，具有强大的抗过拟合能力。其 F1-Score 为 73.09%，并且其精确率 (73.38%) 处于较高水平，结合最低的假阳性，意味着其误报风险最低，预测结果高度可靠。其缺点在于召回率（72.81%）和 F1-Score 略低于 XGB+MLP，性能上限受限于其并行决策树的机制。

2. NaiveBayes 和 Softmax

Softmax 和 NaiveBayes 的性能差距是本次评估中最大的分水岭。

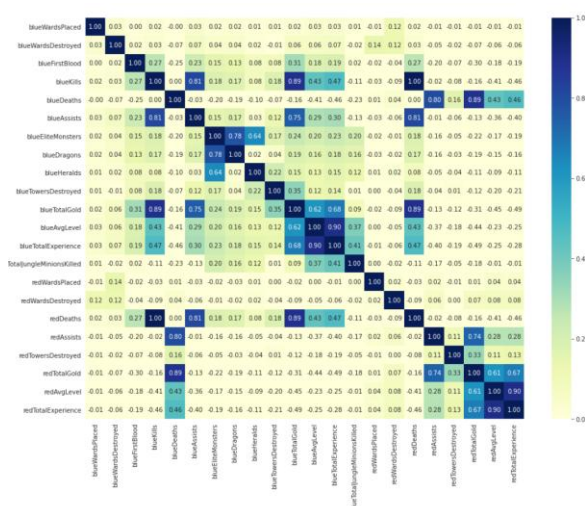
Softmax 的优点是模型相对简单且训练快速，作为具有非线性转换的基准模型，其 F1-Score 为 73.41%，表现优于 XGBoost 和 RandomForest。值得注意的是，其 AUC 甚至略高于所有集成模型（0.817），表明其整体排序区分能力最佳。其缺点是召回率和精确率不如集成模型，性能上限受其浅层结构限制。

NaiveBayes 的优点在于训练速度极快，是所有模型中精确率最高（75.93%）的，意味着它一旦预测为正类，结果非常可靠。然而，其缺点是性能受限，作为唯一的线性概率模型，因特征条件独立性假设被打破而导致召回率极低（59.42%），假阴性数量（403 例）最高，F1-Score 仅为 66.66%，在本任务中几乎不可用。

本次评估展示了非线性模型的绝对优势。以 XGB+MLP、RandomForest 和 Softmax 为代表的非线性方法，在所有关键指标上均大幅领先。这有力地证明了数据特征之间存在高度非线性关系。线性模型 NaiveBayes 的失败证明了数据结构不满足简单模型的假设，其低召回率使其在注重避免漏检的应用中风险极高，不适合本任务。

3. 优化改进

先来看看数据热力图



这张图说明: blueKills 与 redDeaths 之间以及 redKills 与 blueDeaths 之间存在极强的**正相关**(亮红色)。这是游戏规则决定的: 一方的击杀就是另一方的死亡。这些特征是高度冗余的。在建模时, 可以仅保留其中一个作为特征(如只保留 blueKills 和 redKills), 或者只使用其差异(如 $\text{blueKills} - \text{redKills}$)。

blueGoldDiff 与 blueTotalGold、blueTotalExperience、blueKills 等蓝队优势指标之间存在强烈的正相关。blueGoldDiff 作为一个衍生特征，高度浓缩了其他多个特征的信息。如果使用 blueGoldDiff，它已经包含了大部分的经济信息，可以考虑减少对 blueTotalGold 等原始特征的依赖。

因此在代码里面，使用了自定义的 `feature_engineering` 函数，重点是构造差值特征，还要移除冗余的红方特征

根据以上这些，可以进一步分析特征，我们可以做特征工程如下：

```
# --- 1. 构造更有意义的特征 ---  
df['wardsPlacedDiff'] = df['blueWardsPlaced'] - df['redWardsPlaced']  
df['wardsDestroyedDiff'] = df['blueWardsDestroyed'] -  
df['redWardsDestroyed']
```

```

df['blueKDA'] = (df['blueKills'] + df['blueAssists']) /
(df['blueDeaths'] + 1e-5)

df['redKDA'] = (df['redKills'] + df['redAssists']) /
(df['redDeaths'] + 1e-5)

df['KDADiff'] = df['blueKDA'] - df['redKDA']

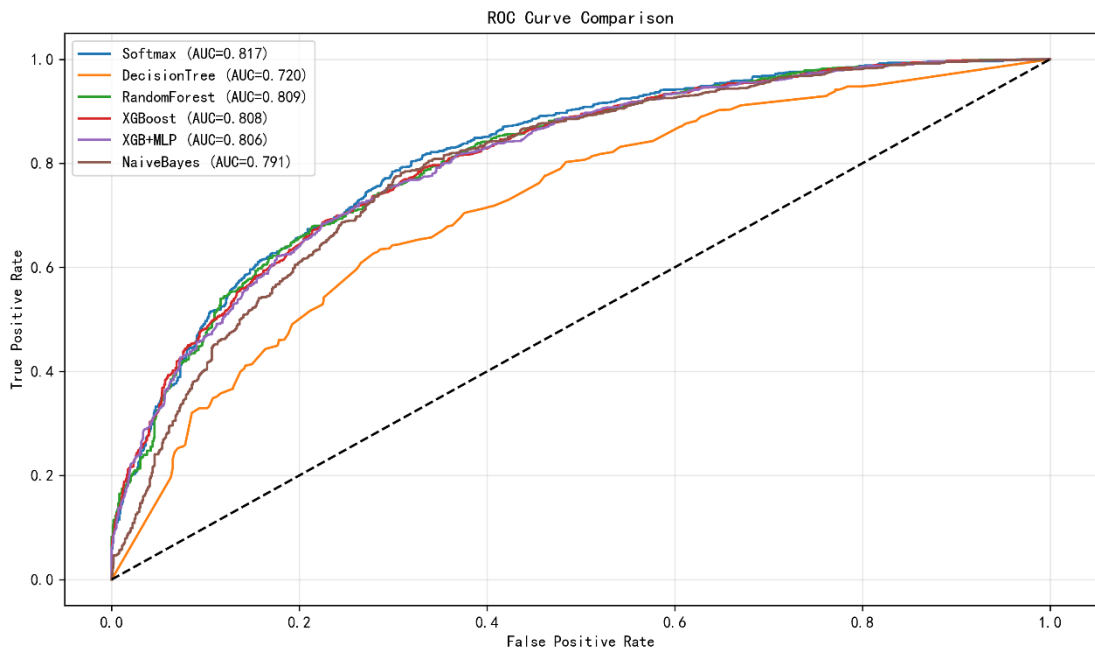
df['jungleDiff'] = df['blueTotalJungleMinionsKilled'] -
df['redTotalJungleMinionsKilled']
.....略

# --- 2. 移除冗余特征 ---

cols_to_drop = [c for c in df.columns if c.startswith('red')]
cols_to_drop.append('gameId')

```

这样的效果更好一些，结果如下，这里为了对比还加入了手写决策树和没有 MLP 优化的 SGBost，可以看出别的相差确实不大，但是朴素贝叶斯相差很大（从 0.65 提升到了 0.791），这就是因为去掉了非线性因素导致的



3. 总结

本项目针对“英雄联盟钻石局十分钟数据胜负预测”这一分类任务，成功搭建并对比了基于生成模型的朴素贝叶斯、基于判别模型的 Softmax 回归，以及高性能的集成方法随机森林、XGBoost 和 XGBoost+MLP 混合模型的性能。

1. 数据特性与预处理的关键性

数据预处理：为了消除特征间量纲差异带来的影响，项目对数值型特征应用了 Z-Score 标准化，确保所有特征尺度一致（均值为 0，标准差为 1）。同时，严格在训练集上拟合参数后应用于测试集，避免了数据泄露。

原始数据挑战： 原始数据集存在特征的高度相关性与共线性（如 blueKills 与 redDeaths），以及特征条件独立性假设被打破的风险。这直接导致了 NaiveBayes 模型在未经优化的原始数据上表现极差（AUC 仅 0.650），验证了该模型不适合处理具有复杂依赖关系的数据结构。

关键优化（特征工程）： 通过分析热力图，项目引入了自定义的特征工程，重点是构造差值特征（如 KDADiff）并移除冗余的红方特征。这一优化是本次实验中提升简单模型性能的最关键步骤。优化后，朴素贝叶斯的 AUC 从 0.650 大幅提升至 0.791，充分证明了特征工程在处理强共线性问题上的有效性。

总之，最终结论是：对于本任务，Softmax 回归模型凭借其出色的泛化能力（训练/测试准确率最为接近）和最高的 AUC 值（0.817），展现出最佳的综合区分能力。集成模型（XGBoost 和 XGB+MLP）虽然在训练集上性能更强，但存在一定的过拟合现象。

2. 思考

正则化参数（ L_2 正则化强度 C ）： Softmax 回归模型采用了 L_2 正则化并结合 K 折交叉验证来确定最优超参数 C ，以应对过拟合问题。正则化参数直接决定了模型抑制过拟合和收缩参数的强度。过大的正则化可能导致欠拟合，而过小则可能导致过拟合，影响模型的泛化能力。

XGBoost 的优势： XGBoost 在目标函数中加入了 L_2 正则化项来惩罚模型的复杂度（即树的叶子节点数量和叶子节点上的得分），有效控制了过拟合，这也是其优于传统 GBDT 的关键，并在本次实验中性能优于随机森林。

非线性与集成学习的局限： 尽管 XGBoost 和 XGB+MLP 理论上更强大，但它们在本任务中表现出更高的过拟合趋势（训练准确率远高于测试准确率）。这表明对于该数据集，过度追求模型的复杂性和拟合能力，可能会牺牲泛化性能，模型的选择需要根据数据的复杂度和泛化性仔细权衡。

4. 组员及任务分配情况

2351274 王亦玮 数据处理+报告撰写+朴素贝叶斯实现+本次 pre

2350233 叶 辰 数据处理+完成了决策树模型+随机森林模型+softmax 回归

2350231 高柏舟 数据可视化+完成了 XGBoost 模型和 XGBoost+MLP 模型训练